Architecture multicores – M2 ESTel

Fiche de révision complète + corrigé guidé du rattrapage 2024

Table des matières

1 Performance : formules essentielles **3**
- 1.1 Speedup . . . 3
- 1.2 CPI . . . 3

2 Pipeline **4**
- 2.1 Sans pipeline . . . 4
- 2.2 Avec pipeline . . . 4
- 2.3 Les trois aléas . . . 4

3 Dépendances : RAW, WAR, WAW **4**
- 3.1 RAW – Read After Write . . . 4
- 3.2 WAR – Write After Read . . . 5
- 3.3 WAW – Write After Write . . . 5
- 3.4 Solution : renommage . . . 5

4 Superscalaire **5**

5 VLIW **5**

6 Superscalaire vs VLIW **6**

7 Multiflot, multithreading et multicore **6**
- 7.1 Parallélisme intra-flot . . . 6
- 7.2 Multiflot . . . 6
- 7.3 Multicore . . . 6

8 Cache **6**
- 8.1 Localité temporelle . . . 6
- 8.2 Localité spatiale . . . 6
- 8.3 Types de cache miss . . . 6
- 8.4 Remèdes . . . 7

9 Transformations de code **7**
- 9.1 Loop Interchange . . . 7
- 9.2 Loop Fusion . . . 7
- 9.3 Loop Unrolling . . . 7
- 9.4 Loop Tiling . . . 8

10 DFG – Data Flow Graph **8**

11 Table de réservation des ressources **8**

12 Filtre d'image du cours **8**
- 12.1 Pourquoi $(2n+1)^2$? . . . 9
- 12.2 Cas $n = 1$ . . . 9
- 12.3 Nombre d'opérations . . . 10

12.4 Cas $n = 2$ . . . 10

13 Exercice 1 du rattrapage 2024 **10**
13.1 1.1 – Différence VLIW / superscalaire . . . 10
13.2 1.2 – Deux situations qui nuisent au parallélisme . . . 11
13.3 1.3 – Multiflot possible ? . . . 11
13.4 1.4 – Technique de réécriture . . . 11
13.5 1.5 – Techniques d'estimation de consommation . . . 11
13.6 1.6 – Parallélisme et consommation . . . 12

14 Exercice 2 du rattrapage 2024 **12**
14.1 2.1 – Nombre maximal d'instructions parallèles . . . 12
14.2 2.2 – Loi du courant . . . 12
14.3 2.3 – Campagne de mesure . . . 13

15 Exercice 3 – Architecture **13**

16 Exercice 3 – Réseau 1 **14**

17 Exercice 3 – Réseau 2 **14**

18 Exercice 3 – Réseau 3 **15**

19 3.1 – Temps d'exécution idéal **15**
19.1 Borne arithmétique du réseau 1 . . . 16
19.2 Borne réseau 2 . . . 16
19.3 Borne réseau 3 . . . 16

20 3.2 – Nouvelle écriture **17**

21 3.3 – Multiflot **17**

22 3.4 – Comparaison **18**

23 3.5 – Puissance et énergie **18**
23.1 Énergie d'un cycle . . . 18
23.2 Énergie totale . . . 19
23.3 Méthode question 3.5 . . . 19

24 Puissance vs énergie **19**

25 Temps réel et multicore **20**

26 Checklist examen **20**

27 Questions flash **21**

1 Performance : formules essentielles

COURS – à connaître

La performance est liée au temps d'exécution :

$$\boxed{\text{Performance} = \frac{1}{T_{\text{exe}}}}$$

Le temps d'exécution peut être exprimé en nombre de cycles :

$$\boxed{T_{\text{exe}} = N_{\text{cycles}} T_{\text{cycle}}}$$

Or :

$$\boxed{T_{\text{cycle}} = \frac{1}{f}}$$

donc :

$$\boxed{T_{\text{exe}} = \frac{N_{\text{cycles}}}{f}}$$

À :

$$f = 1 \text{ GHz}$$

on a :

$$\boxed{T_{\text{cycle}} = 1 \text{ ns}}$$

Exemple :

$$12 \text{ cycles} \Rightarrow 12 \text{ ns}$$

1.1 Speedup

Pour comparer deux architectures :

$$\boxed{S = \frac{T_{\text{ancien}}}{T_{\text{nouveau}}}}$$

Exemple :

$$T_1 = 20\ ns, \qquad T_2 = 10\ ns$$

alors :

$$S = \frac{20}{10} = 2$$

L'architecture 2 est donc deux fois plus rapide.

1.2 CPI

$$\boxed{CPI = \frac{N_{\text{cycles}}}{N_{\text{instructions}}}}$$

Plus le CPI est faible, plus le processeur termine d'instructions par cycle.

2 Pipeline

Le cours décompose classiquement l'exécution d'une instruction en :

$$\boxed{IF \rightarrow DEC \rightarrow EX \rightarrow MEM \rightarrow WB}$$

avec :
— IF : Instruction Fetch ;
— DEC : décodage ;
— EX : exécution ;
— MEM : accès mémoire ;
— WB : Write Back.

2.1 Sans pipeline

Chaque instruction traverse toutes les étapes avant le lancement de la suivante.
Avec un pipeline à k étages :

$$\boxed{T_{\text{non pipeline}} = N \times k \times T_{\text{cycle}}}$$

dans le modèle idéal.

2.2 Avec pipeline

Les instructions se chevauchent.
Pour N instructions dans un pipeline de k étages :

$$\boxed{T_{\text{pipeline}} = (k + N - 1)T_{\text{cycle}}}$$

dans le cas idéal.
Pour N grand :

$$\boxed{\text{débit} \rightarrow 1 \text{ instruction/cycle}}$$

2.3 Les trois aléas

1. **Aléa structurel**
   Deux instructions demandent la même ressource pendant le même cycle.
2. **Aléa de données**
   Une instruction dépend du résultat d'une autre.
3. **Aléa de contrôle**
   Branches :

$$IF, \quad JUMP, \quad boucle, \quad BNE, \quad BEQ...$$

3 Dépendances : RAW, WAR, WAW

3.1 RAW – Read After Write

$$\boxed{RAW = \text{vraie dépendance}}$$

Exemple :

$$R1 = R2 + R3$$

puis :

$$R4 = R1 + R5$$

La seconde instruction doit attendre la production de $R1$.

3.2 WAR – Write After Read

Une instruction écrit dans un registre qu'une instruction précédente doit encore lire.

$$\boxed{WAR = \text{fausse dépendance}}$$

3.3 WAW – Write After Write

Deux instructions écrivent dans le même registre.

$$\boxed{WAW = \text{fausse dépendance}}$$

3.4 Solution : renommage

Le cours présente l'utilisation de registres physiques et d'un ROB :

$$\boxed{ROB = \text{ReOrder Buffer}}$$

Le renommage permet de supprimer principalement les fausses dépendances :

$$WAR, \qquad WAW$$

4 Superscalaire

COURS – à connaître

Dans une architecture superscalaire :

$$\boxed{\text{le matériel détecte et exploite le parallélisme}}$$

Le processeur doit :
— disposer de plusieurs unités fonctionnelles ;
— trouver les instructions indépendantes ;
— vérifier les dépendances ;
— répartir les instructions sur les unités disponibles.

Architecture superscalaire de degré n :

$$\boxed{\text{jusqu'à } n \text{ instructions terminées par cycle}}$$

dans le cas idéal.

5 VLIW

COURS – à connaître

VLIW :

$$\boxed{\text{Very Long Instruction Word}}$$

Le compilateur doit présenter au processeur des instructions pouvant être exécutées en parallèle.
Le parallélisme est donc organisé :

$$\boxed{\text{statiquement à la compilation}}$$

Le VLIW utilise :
— ordonnancement statique ;
— graphe de dépendances ;
— plusieurs unités fonctionnelles ;
— exécution parallèle ;

— éventuellement exécution pipeline.

6 Superscalaire vs VLIW

	Superscalaire	VLIW
Qui trouve le parallélisme ?	Le matériel	Le compilateur
Moment	Pendant l'exécution	À la compilation
Ordonnancement	Dynamique	Statique
Dépendances	Gérées par le matériel	Analysées par le compilateur
Complexité matériel	Plus importante	Plus faible

7 Multiflot, multithreading et multicore

7.1 Parallélisme intra-flot

Superscalaire et VLIW exploitent principalement :

$$\boxed{ILP = \text{Instruction Level Parallelism}}$$

donc plusieurs instructions indépendantes d'un même flot.

7.2 Multiflot

Le multiflot correspond à l'exécution de plusieurs :

$$\boxed{\text{threads / tâches / flots}}$$

7.3 Multicore

Dans le cas multicore :

$$\boxed{\text{l'OS répartit les tâches sur les différents cœurs}}$$

8 Cache

8.1 Localité temporelle

Une donnée utilisée récemment a de fortes chances d'être réutilisée prochainement.

8.2 Localité spatiale

Après une adresse mémoire donnée, les adresses voisines ont de fortes chances d'être utilisées.
Exemple :

$$A[i], A[i+1], A[i+2], ...$$

8.3 Types de cache miss

1. **Obligatoire**
   Premier accès à une ligne.
2. **Capacité**
   Le cache est trop petit.
3. **Conflit**
   Plusieurs blocs utilisent la même zone du cache.

8.4 Remèdes

Défaut	Remède
Obligatoire	ligne de cache plus grande
Capacité	cache plus grand / tiling
Conflit	associativité plus élevée

9 Transformations de code

Le cours présente plusieurs optimisations.

9.1 Loop Interchange

Changer l'ordre des boucles :

$$for(i) \quad for(j)$$

devient :

$$for(j) \quad for(i)$$

Objectif :

$$\boxed{\text{améliorer la localité mémoire}}$$

9.2 Loop Fusion

Avant :

$$for(i) \quad B[i] = A[i] + 1$$

puis :

$$for(i) \quad C[i] = 3A[i]$$

Après :

$$for(i) \begin{cases} B[i] = A[i] + 1 \\ C[i] = 3A[i] \end{cases}$$

Gain :

$$\boxed{\text{moins d'accès mémoire}}$$

9.3 Loop Unrolling

Déplier plusieurs itérations.
Avant :

$$for(i) \quad B[i] = A[i] + A[i+1]$$

Après :
plusieurs opérations sont mises dans le corps de boucle.
Gains :
— moins de branches ;
— moins de contrôle de boucle ;
— davantage d'instructions indépendantes ;
— davantage d'ILP.
Inconvénient :

$$\boxed{\text{taille du code augmente}}$$

9.4 Loop Tiling

Découpage des matrices en blocs :

blocs suffisamment petits pour tenir dans le cache

10 DFG – Data Flow Graph

Le DFG représente :

— les opérations ;
— les dépendances entre opérations.

Exemple :

$$A = B \times C + D$$

DFG :

$$B, C \rightarrow MULT \rightarrow ADD \rightarrow A$$

avec D directement relié au ADD.

11 Table de réservation des ressources

Quand une architecture possède par exemple :

$$2\ MEM, \quad 2\ ADD, \quad 1\ MUL$$

il faut remplir les ressources cycle par cycle.

Exemple :

Cycle	MEM1	MEM2	ADD1	ADD2	MUL
1	Load A	Load B	–	–	–
2	Load C	–	–	–	A×B
3	–	–	AB+C	–	–

MÉTHODE

Toujours faire :
1. liste des opérations ;
2. dépendances ;
3. ressources disponibles ;
4. remplissage cycle par cycle ;
5. dernier cycle = temps d'exécution idéal.

12 Filtre d'image du cours

Le filtre est :

$$g(x,y) = \sum_{i=-n}^{n} \sum_{j=-n}^{n} h(i,j) I(x+i, y+j)$$

Le nombre de pixels est :

$$\boxed{N = (2n+1)^2}$$

12.1 Pourquoi $(2n+1)^2$?

i prend :

$$-n, \ldots, -1, 0, 1, \ldots, n$$

donc :

$$2n+1$$

valeurs.
Même chose pour j.
Ainsi :

$$N = (2n+1)(2n+1)$$

donc :

$$\boxed{N = (2n+1)^2}$$

12.2 Cas $n = 1$

$$N = (2 \times 1 + 1)^2$$

$$N = 3^2$$

$$\boxed{N = 9}$$

On considère donc un masque :

$$3 \times 3$$

Le masque donné dans le cours est :

$$h = \frac{1}{10} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Pourquoi $2I_5$?
Si :

$$\begin{matrix} I_1 & I_2 & I_3 \\ I_4 & I_5 & I_6 \\ I_7 & I_8 & I_9 \end{matrix}$$

alors :

$$g(x,y) = \frac{1}{10}(I_1 + I_2 + I_3 + I_4 + 2I_5 + I_6 + I_7 + I_8 + I_9)$$

Le 10 vient de :

$$1+1+1+1+2+1+1+1+1 = 10$$

donc :

$$\boxed{10 = \text{somme des coefficients}}$$

et non nombre de pixels.

12.3 Nombre d'opérations

Forme générale :

$$9\ MULT$$

et pour additionner 9 résultats :

$$9 - 1 = 8\ ADD$$

donc :

$$\boxed{9\ MULT + 8\ ADD}$$

hors accès mémoire.

12.4 Cas $n = 2$

$$N = (2 \times 2 + 1)^2$$

$$N = 5^2$$

$$\boxed{N = 25}$$

Donc forme générale :

$$25\ MULT$$

et :

$$25 - 1 = 24\ ADD$$

$$\boxed{25\ MULT + 24\ ADD}$$

13 Exercice 1 du rattrapage 2024

13.1 1.1 – Différence VLIW / superscalaire

RÉPONSE TYPE EXAMEN

Une architecture superscalaire détecte dynamiquement au niveau matériel les instructions pouvant être exécutées en parallèle.
Une architecture VLIW repose sur le compilateur, qui regroupe statiquement dans une même longue instruction les opérations pouvant être exécutées simultanément.

$$\boxed{\text{Superscalaire} = \text{matériel}}$$

$$\boxed{\text{VLIW} = \text{compilateur}}$$

13.2 1.2 – Deux situations qui nuisent au parallélisme

RÉPONSE TYPE EXAMEN

Deux situations classiques :
1. **Dépendance de données**
   Par exemple une RAW : une instruction doit attendre le résultat de l'instruction précédente.
2. **Branchement**
   Le processeur ne connaît pas immédiatement le prochain chemin du programme.

Ces situations empêchent certaines instructions d'être exécutées simultanément.

On peut aussi citer :

conflit de ressources

13.3 1.3 – Multiflot possible ?

RÉPONSE TYPE EXAMEN

Le fait qu'un processeur soit superscalaire ou VLIW ne signifie pas automatiquement qu'il est multiflot.
Superscalaire et VLIW exploitent principalement le parallélisme entre instructions d'un même flot.
Pour exécuter plusieurs flots simultanément, il faut une architecture supportant plusieurs threads ou plusieurs contextes d'exécution, ou plusieurs cœurs.

13.4 1.4 – Technique de réécriture

RÉPONSE TYPE EXAMEN

Une technique adaptée est :

Loop Unrolling

Le dépliage de boucle :
— réduit le nombre de branches ;
— réduit les instructions de contrôle ;
— expose davantage d'instructions indépendantes ;
— permet de mieux exploiter une architecture superscalaire ou VLIW.

13.5 1.5 – Techniques d'estimation de consommation

ATTENTION

Le sujet demande explicitement cette question, mais le document M2_archi_P1 fourni ne contient pas, dans les passages disponibles, la définition suffisamment claire des deux techniques.
Il ne faut donc pas présenter une réponse comme provenant du cours sans disposer de la partie consommation correspondante.

Réponse académique généralement rencontrée :
— estimation par **modélisation / simulation** ;
— estimation par **mesure expérimentale**.

Principe :
— modèle : estimer la consommation à partir de l'activité des blocs ou instructions ;
— mesure : mesurer courant et tension sur la plateforme réelle.

HYPOTHÈSE NÉCESSAIRE

Cette formulation est cohérente avec le sujet, mais doit être vérifiée avec la partie du cours consacrée explicitement à la consommation.

13.6 1.6 – Parallélisme et consommation

Le parallélisme active plusieurs unités simultanément.
Ainsi la puissance instantanée peut augmenter.
En revanche, le temps d'exécution peut diminuer.
Comme :

$$\boxed{E = P \times T}$$

l'effet sur l'énergie dépend des deux phénomènes :

$$P \uparrow$$

mais :

$$T \downarrow$$

RÉPONSE TYPE EXAMEN

En général, davantage de parallélisme peut augmenter la puissance instantanée car davantage d'unités fonctionnent simultanément.
Cependant, le temps d'exécution est réduit. Il faut donc distinguer puissance et énergie.

14 Exercice 2 du rattrapage 2024

Architecture :

$$2\ INT$$

$$1\ MULT/DIV$$

$$2\ FLOAT$$

$$4\ MEM$$

14.1 2.1 – Nombre maximal d'instructions parallèles

Si toutes les ressources sont indépendantes et toutes utilisées :

$$N_{\max} = 2 + 1 + 2 + 4$$

$$\boxed{N_{\max} = 9}$$

RÉPONSE TYPE EXAMEN

Le cœur peut théoriquement exécuter jusqu'à :

$$\boxed{\text{9 opérations simultanément}}$$

à condition que les instructions soient indépendantes et que toutes les unités soient alimentées.

14.2 2.2 – Loi du courant

Le sujet donne :

$$I = I_0 + iI_1 + jI_2 + kI_3 + lI_4$$

avec :

$$i = \text{nombre d'opérations entières}$$

$$j = \text{MULT/DIV}$$

$$k = \text{opérations flottantes}$$

$$l = \text{accès mémoire}$$

Donc :

$$\boxed{0 \leq i \leq 2}$$

$$\boxed{0 \leq j \leq 1}$$

$$\boxed{0 \leq k \leq 2}$$

$$\boxed{0 \leq l \leq 4}$$

Interprétation :

— I_0 : courant de base ;
— I_1 : surcoût d'une opération entière ;
— I_2 : surcoût MULT/DIV ;
— I_3 : surcoût d'une opération flottante ;
— I_4 : surcoût d'un accès mémoire.

14.3 2.3 – Campagne de mesure

RÉPONSE TYPE EXAMEN

Créer plusieurs micro-benchmarks.
Exemple :
1. mesurer le courant au repos : obtention de I_0 ;
2. activer une unité entière ; mesurer l'augmentation ;
3. activer deux unités entières ;
4. répéter pour MULT/DIV ;
5. répéter pour flottants ;
6. répéter pour les accès mémoire ;
7. combiner les unités afin de vérifier le modèle additif.

On identifie ensuite :

$$I_0,\ I_1,\ I_2,\ I_3,\ I_4$$

à partir des mesures.

15 Exercice 3 – Architecture

Le cœur superscalaire fonctionne à :

$$\boxed{f = 1\ GHz}$$

donc :

$$\boxed{T_{\text{cycle}} = 1\ ns}$$

Le parallélisme donné est :

$$2\ INT$$

$$2\ FLOAT$$

$$3\ MEM$$

Le réseau possède :

$$4 \text{ sorties intermédiaires}$$

$$Y(1), Y(2), Y(3), Y(4)$$

puis :

$$S(1), S(2), S(3)$$

16 Exercice 3 – Réseau 1

Pour :

$$i = 1, \ldots, 4$$

le sujet donne :

$$Y(i) = W_1(i,1)I(1) + W_1(i,2)I(2) + W_1(i,3)I(3) + W_1(i,4)I(4)$$

Pour un $Y(i)$:

$$4\ MULT$$

et :

$$3\ ADD$$

Donc pour les quatre neurones :

$$\boxed{16\ MULT}$$

et :

$$\boxed{12\ ADD}$$

Les quatre neurones :

$$Y(1), Y(2), Y(3), Y(4)$$

sont indépendants les uns des autres.
C'est donc une source importante de parallélisme.

17 Exercice 3 – Réseau 2

Le sujet donne deux sorties :

$$S(1), S(2)$$

sous forme de sommes pondérées des sorties du premier réseau.
Chaque sortie contient quatre produits.
Donc pour les deux sorties :

$$\boxed{8\ MULT}$$

et :

$$\boxed{6\ ADD}$$

mais elles dépendent de :

$$Y(1), Y(2), Y(3), Y(4)$$

Donc :

$$\boxed{\text{RAW entre Réseau 1 et Réseau 2}}$$

18 Exercice 3 – Réseau 3

Le sujet donne :

$$S(3) = W_4(1)Y(1) + W_4(2)Y(3) + W_4(3)I(5) + W_4(4)I(6)$$

Donc :

$$\boxed{4\ MULT + 3\ ADD}$$

S3 dépend seulement de :

$$Y(1), Y(3), I(5), I(6)$$

Il n'a donc pas besoin d'attendre nécessairement toute la fin du Réseau 1.
Dès que :

$$Y(1)$$

et :

$$Y(3)$$

sont disponibles, le Réseau 3 peut commencer.

19 3.1 – Temps d'exécution idéal

ATTENTION

Le sujet donne :

$$2\ FLOAT$$

mais il ne précise pas explicitement, dans la page fournie, si une multiplication flottante et une addition flottante peuvent être réalisées chacune en un cycle, ni la latence exacte des unités.
On ne peut donc pas donner un nombre de cycles unique sans poser cette hypothèse.

HYPOTHÈSE NÉCESSAIRE

Pour construire une table de réservation d'entraînement, on peut supposer :

$$1\ FLOAT = 1\ cycle$$

et :

$$1\ MEM = 1\ cycle$$

avec résultat disponible au cycle suivant.

Sous cette hypothèse :

l'architecture peut faire au maximum :

$$\boxed{2\ opérations flottantes par cycle}$$

et :

$$\boxed{3\ accès mémoire par cycle}$$

19.1 Borne arithmétique du réseau 1

Réseau 1 :

$$16\ MULT + 12\ ADD$$

soit :

$$28\ FLOAT$$

Avec 2 unités flottantes :

$$\boxed{T_{\min} \geq \left\lceil \frac{28}{2} \right\rceil = 14\ cycles}$$

Cette valeur est une borne.
Les dépendances internes des additions peuvent empêcher d'atteindre exactement cette borne.

19.2 Borne réseau 2

Réseau 2 :

$$8\ MULT + 6\ ADD$$

$$14\ FLOAT$$

donc :

$$\boxed{T_{\min} \geq 7\ cycles}$$

après disponibilité des Y nécessaires.

19.3 Borne réseau 3

$$4\ MULT + 3\ ADD$$

$$7\ FLOAT$$

donc :

$$\boxed{T_{\min} \geq 4\ cycles}$$

avec deux unités flottantes.

20 3.2 – Nouvelle écriture

La première écriture utilise des boucles.
Une meilleure écriture peut exposer simultanément les calculs de plusieurs neurones.
Exemple conceptuel :

$$Y_1, \quad Y_2$$

peuvent être calculés simultanément.
Puis :

$$Y_3, \quad Y_4$$

RÉPONSE TYPE EXAMEN

Oui, une nouvelle écriture peut améliorer le temps d'exécution.
Il faut exposer davantage de parallélisme en calculant plusieurs neurones indépendants simultanément.
On peut utiliser notamment :

Loop Unrolling

Les gains attendus sont :
— meilleure utilisation des deux unités flottantes ;
— réduction du nombre de branches ;
— davantage d'ILP ;
— diminution du temps d'exécution.

21 3.3 – Multiflot

Le graphe montre deux branches après Réseau 1 :

Réseau 2

et :

Réseau 3

On peut donc définir par exemple :

$$P_1 = \text{calcul des } Y$$

$$P_2 = \text{calcul de } S_1, S_2$$

$$P_3 = \text{calcul de } S_3$$

avec :

$$P_2$$

et :

$$P_3$$

pouvant fonctionner en parallèle lorsque leurs entrées sont disponibles.

RÉPONSE TYPE EXAMEN

Oui, le multiflot est possible.
Après le calcul des valeurs Y nécessaires, les calculs de :

$$S(1), S(2)$$

et :

$$S(3)$$

appartiennent à des branches indépendantes.
On peut donc créer plusieurs processus/threads.
Le gain attendu est la diminution du temps total si plusieurs ressources ou plusieurs cœurs sont disponibles.

22 3.4 – Comparaison

Pour comparer deux architectures :

$$\boxed{S = \frac{T_1}{T_2}}$$

Une architecture avec davantage de ressources :

— exécute davantage d'opérations simultanément ;
— réduit potentiellement le nombre de cycles ;
— augmente généralement la puissance instantanée.

RÉPONSE TYPE EXAMEN

L'architecture offrant le plus de ressources parallèles peut réduire le temps d'exécution, mais seulement si le programme présente suffisamment d'opérations indépendantes.
Le gain réel est donc limité par :
— les dépendances ;
— les accès mémoire ;
— le nombre d'unités disponibles ;
— le parallélisme réellement extrait.

23 3.5 – Puissance et énergie

Le sujet donne :

N_{instr}	1	2	3	4	5	6	7	8
P_{core}(mW)	80	100	120	140	160	180	190	200

Et :

$$f = 1\ GHz$$

donc :

$$\boxed{1\ cycle = 1\ ns}$$

23.1 Énergie d'un cycle

$$E = P \times T$$

Exemple avec :

$$4\ instructions$$

on lit :

$$P = 140\ mW$$

pendant :

$$1\ ns$$

alors :

$$E = 140 \times 10^{-3} \times 10^{-9}$$

$$E = 140 \times 10^{-12}\ J$$

$$\boxed{E = 140\ pJ}$$

Donc à 1 GHz :

$$\boxed{E_{\text{cycle}}(pJ) = P_{\text{cycle}}(mW)}$$

numériquement.

23.2 Énergie totale

Pour chaque cycle :

$$E_k = P_k T_{\text{cycle}}$$

L'énergie totale vaut :

$$\boxed{E_{\text{tot}} = \sum_k P_k T_{\text{cycle}}}$$

À 1 GHz :

$$\boxed{E_{\text{tot}}(pJ) = \sum_k P_k(mW)}$$

si chaque ligne correspond à un cycle.

23.3 Méthode question 3.5

MÉTHODE

Pour chaque cycle de la table de réservation :
1. compter le nombre total d'instructions exécutées ;
2. lire la puissance correspondante dans le tableau ;
3. multiplier par 1 ns ;
4. additionner toutes les énergies.

24 Puissance vs énergie

Ne jamais confondre :

$$\boxed{P = \text{puissance}}$$

et :

$$\boxed{E = \text{énergie}}$$

La puissance est une consommation instantanée.
L'énergie dépend de :

$$\boxed{E = P \times T}$$

Le parallélisme peut faire :

$$P \uparrow$$

mais :

$$T \downarrow$$

Donc l'énergie peut :

$$\uparrow, \quad \downarrow, \quad \text{ou rester proche}$$

selon l'architecture.

25 Temps réel et multicore

Pour une tâche i :

$$C_i = WCET$$

$$P_i = \text{période}$$

Charge :

$$\boxed{U_i = \frac{C_i}{P_i}}$$

Charge totale :

$$\boxed{U = \sum_i \frac{C_i}{P_i}}$$

Sur un cœur :
si :

$$U > 1$$

alors la charge demandée est supérieure à 100%.

26 Checklist examen

— Performance = $1/T$
— $T = N_{\text{cycles}}/f$
— $T_{\text{cycle}} = 1/f$
— Speedup = T_1/T_2
— Pipeline : IF–DEC–EX–MEM–WB
— Aléa structurel
— Aléa de données
— Aléa de contrôle
— RAW = vraie dépendance
— WAR = fausse dépendance
— WAW = fausse dépendance
— Superscalaire = matériel
— VLIW = compilateur
— ILP = intra-flot
— Multiflot = plusieurs flots
— Multicore = OS répartit tâches
— Loop Interchange
— Loop Fusion
— Loop Unrolling
— Loop Tiling
— Localité temporelle
— Localité spatiale
— Cache miss obligatoire
— Cache miss capacité
— Cache miss conflit
— Toujours dessiner le DFG

— Toujours faire table de réservation
— Respecter type des unités
— Filtre : $(2n+1)^2$
— Addition de N valeurs : $N-1$ ADD
— $P = UI$
— $E = PT$
— À 1 GHz : 1 cycle = 1 ns

27 Questions flash

1. Qui exploite le parallélisme en superscalaire ?
2. Qui exploite le parallélisme en VLIW ?
3. Qu'est-ce qu'une RAW ?
4. Pourquoi un branchement ralentit-il le pipeline ?
5. À quoi sert le loop unrolling ?
6. Quelle différence entre ILP et multiflot ?
7. Pour $n = 1$, combien de pixels ?
8. Pour $n = 2$, combien de pixels ?
9. Combien d'ADD faut-il pour additionner 25 valeurs ?
10. Une architecture a : 2 INT, 1 MUL, 2 FLOAT, 4 MEM. Combien d'opérations maximum ?
11. À 1 GHz, combien vaut un cycle ?
12. 160 mW pendant 1 ns représentent combien de pJ ?

Réponses questions flash

1. Le matériel.
2. Le compilateur.
3. Read After Write : vraie dépendance.
4. Le prochain chemin d'exécution dépend du résultat de la condition.
5. Exposer davantage d'ILP et réduire le contrôle de boucle.
6. ILP : instructions d'un même flot.
   Multiflot : plusieurs flots/threads.
7. $$(2 \times 1 + 1)^2 = 9$$
8. $$(2 \times 2 + 1)^2 = 25$$
9. $$25 - 1 = 24$$
10. $$2 + 1 + 2 + 4 = 9$$
    sous hypothèse d'indépendance des ressources.
11. $$1\ ns$$
12. $$160\ pJ$$

Résumé final ultra-court

$$\boxed{\text{Superscalaire} = \text{parallélisme dynamique matériel}}$$

$$\boxed{\text{VLIW} = \text{parallélisme statique compilateur}}$$

$$\boxed{RAW = \text{vraie dépendance}}$$

$$\boxed{WAR, WAW = \text{fausses dépendances}}$$

$$\boxed{T = \frac{N_{\text{cycles}}}{f}}$$

$$\boxed{Speedup = \frac{T_1}{T_2}}$$

$$\boxed{N_{\text{filtre}} = (2n+1)^2}$$

$$\boxed{N \text{ valeurs} \Rightarrow N - 1\ ADD}$$

$$\boxed{E = P \times T}$$

$$\boxed{f = 1\ GHz \Rightarrow 1\ cycle = 1\ ns}$$